Assignment 3: Efficient Route Planning

Martin Aumüller and Holger Dell

Hand-in date: November 12, 2025, 11:59

The assignment is to be solved in groups of 1–3 people.

1 Introduction

Your task is to implement a variant of the contraction hierarchies algorithm by Geisberger et al. [GSSV12] for efficient route planning in road networks. With this task comes a real-world graph that depicts the road network in Denmark with $n = 569,586$ vertices and $m = 587,643$ edges. This file is available on learnIT and will be the basis for running experiments.

You will need to test your implementations for correctness, perform experiments where you determine the average search time and the average number of edges that were inspected by the algorithm. Furthermore, you will have to detail your choices on how you implemented contraction hierarchies and report on its preprocessing and search time.

2 Dataset

Your implementation is supposed to read files (through standard input) that have the following structure. The first line contains two integers n, the number of vertices, and m, the number of edges. Each of the following n lines contain three numbers: an integer that identifies the vertex, and two floating point values representing the longitude and latitude of this vertex. Finally, m lines follow representing the edges of the graph. Each line contains three integers: the first two represent the identifiers of the two nodes it connects, and the last one is the cost of the edge. We simplify the setting and assume that the graph is undirected, i.e., does not contain one-way roads.

3 Bidirectional Dijkstra

In this first part, your task is to implement a bidirectional version of Dijkstra's algorithm, as discussed in class. We strongly recommend that you are using the Java code template available at `http://github.itu.dk/algorithms/contraction-hierarchies-template` as a starting point.

Algorithm 1 Bidirectional Dijkstra(s, t)

$d_l, d_r \leftarrow$ empty maps; settled $\leftarrow$ empty set; $d \leftarrow \infty$
$d_l[s] \leftarrow 0, d_r[t] \leftarrow 0$
$Q_l \leftarrow \{(s, 0)\}, Q_r \leftarrow \{(t, 0)\}$ //PQs, l from start, r from target
while $Q_l \neq \emptyset$ or $Q_r \neq \emptyset$ **do**
  if $Q_l \neq \emptyset$ and $\min(Q_l) \leq \min(Q_r)$ **then**
    $i \leftarrow l$
  else
    $i \leftarrow r$
  end if
  $(u, \cdot) \leftarrow Q_i$.deleteMin()
  if u in settled **then**
    //This vertex was settled by the other instance
    break
  end if
  settled.add(u)
  for all $(u, v) \in E$ **do**
    if $d_i[u] + c(u, v) < d_i[v]$ **then**
      $d_i[v] \leftarrow d_i[u] + c(u, v)$
      Update Q_i with $(v, d_i[v])$
    end if
    $d \leftarrow \min(d, d_l[v] + d_r[v])$ //Update minimum dist between s and t
  end for
end while
return d

The readme file in the template discusses the basic functionality, such as reading the graph and running a Dijkstra search.

Solve the following two tasks:

1. Create an implementation of bidirectional Dijkstra, as described in Algorithm 1. Note that we only compute the distance of a (s, t) pair and not recover an actual shortest path between s and t. Add unit tests that test that your implementation produces the correct distances on an example graph.

2. Using a fixed seed, choose 1 000 random (s, t) pairs in the Denmark graph. Report on the average running time and the average number of relaxed edges when searching the 1 000 (s, t) pairs, both for the provided Dijkstra implementation and your bidirectional version. Find a suitable presentation to compare these two implementations with each other.

4 Contraction Hierarchies

In this second part, you will implement Contraction Hierarchies [GSSV12]. We will focus on the variant discussed in class, in which we use the "edge difference" heuristic to set up the contraction order, and use "lazy updates" to update the edge differences during preprocessing. In the following, you are free to modify the API of the provided Java template, i.e., the signature of methods, to include additional parameters that simplify the program logic or make it easier to test. For each method that you implement, you are expected to write unit tests.

Solve the following tasks regarding building the augmented graph, i.e. the graph that contains the original graph and all shortcuts added during preprocessing:

1. Implement a method `getEdgeDifference` that takes the identifier of a vertex v and returns the *edge difference* after contraction, i.e., computes the change in edges by contracting v.

2. Implement a method `contract` that takes the identifier of a vertex v as argument and carries out the contraction of v in the current graph.

3. Implement the *preprocessing* phase of Contraction Hierarchies. Initially, order all vertices by their edge difference. Carry out contractions in the order of computed edge differences. Make sure to use the *lazy update* approach which recomputes the edge difference of vertex v before carrying out the contraction. This ensures that contractions are carried out in the right order.

   Remark: The preprocessing phase can take several minutes to complete on the graph of Denmark. Make sure to include some debugging information that shows the progress when testing.

4. Add a method `storeGraph` that writes the *augmented graph* (original graph + all shortcuts added) that resulted from running the preprocessing phase. The format of the file should be as follows: The first line contains the number of vertices and edges. (The former should be the same as the input graph, the latter is potentially larger due to added shortcuts). Then follow n lines representing the vertices as in the input file, but with an additional integer representing the *rank* of the node, i.e., at which point it was contracted. Finally, m lines follow representing the edges of the graph as in the input file with one additional integer at the end which represents the identifier of the contracted node if it is shortcut, or -1 if it is not a shortcut.

Add to your report the time it took to build the augmented graph for the graph of Denmark. Include the resulting number of edges. Describe your general architectural choices: How do you carry out the Dijkstra searches necessary to decide if a shortcut has to be added? How do you ensure that contracted nodes are no longer used?

In the final part of the assignment, you will make use of this graph to speed up your shortest path searches by solving the following two tasks:

1. Implement the contraction hierarchies bidirectional search algorithm. The algorithm is similar to Algorithm 1, with the only exception that the forward search from s only ever considers edges (u, v) for which the rank is increasing, and the backward search goes up in rank as well. Make sure to add unit tests and verify that the distance values are identical to your previous implementations.

2. Report on the performance of the search on the same 1 000 pairs on the Denmark graph as in Section 3, using the same performance metrics.

5 Visualizing the Augmented Graph

The Github repository `https://github.itu.dk/altp/contraction-hierachy-visualiser` contains a visualization tool. This can be useful to get a visual impression of the correctness of the preprocessing phase. The model solution for the task resulted in the graph shown in Figure 1.

6 Report

Write a report in LaTeX, following the advice given on the lectures and the feedback you got from your report in the first two assignments. The report must not exceed **4 pages**, **excluding possible tables, figures, and references**. Hand in the report and the code you produced as a **single zip file** through LearnIT. Your group only needs to hand in one report.

References

[GSSV12] Robert Geisberger, Peter Sanders, Dominik Schultes, and Christian Vetter. Exact routing in large road networks using contraction hierarchies. *Transp. Sci.*, 46(3):388–404, 2012.

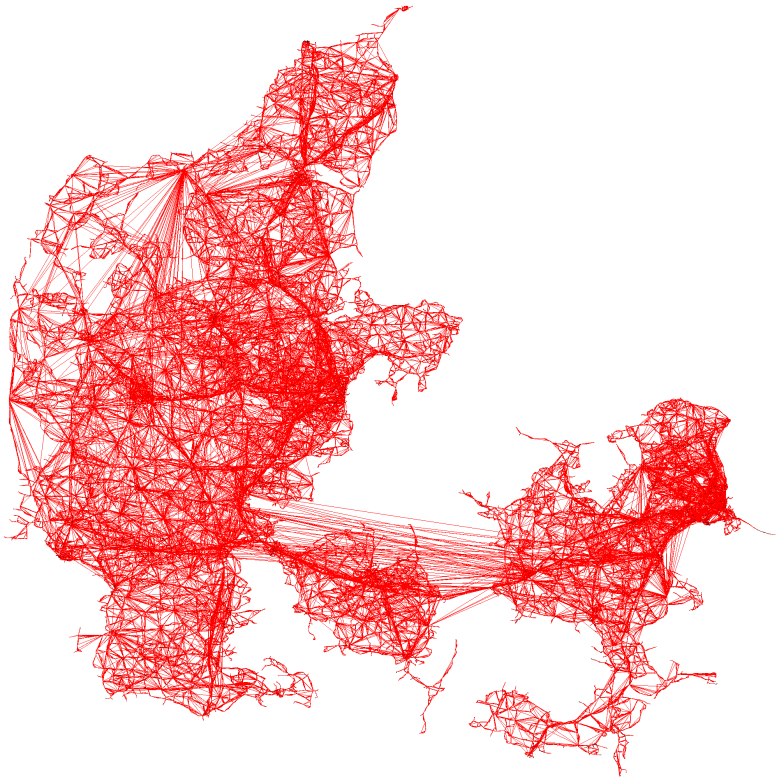


Figure 1: Example of augmented graph for road network of Denmark.

A Bonus tasks

THE FOLLOWING IS NOT PART OF THE MANDATORY ASSIGNMENT.

If you want to explore this topic further, please find some suggestions of additional tasks in the following. Please report on these tasks in your report: These tasks *do not count* towards the page limit.

Proper experimentation of the pre-processing phase. Implement and evaluate at least two different node ordering strategies for the pre-processing phase of contraction hierarchies. Furthermore, evaluate the impact the resulting augmented graphs have for the search performance.

Recover the path. Implement and evaluate an option to recover the shortest path from the shortest path tree that (most likely) include contracted edges.

Higher resolution graph with one way streets. As you have seen in the lecture demo, the graph actually ignores small residential streets. Write a parser to retrieve a more detailed graph from `https://download.geofabrik.de/europe/denmark.html`, and evaluate the performance of your implementation on this larger graph.